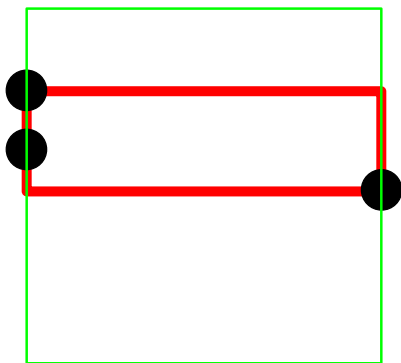


Analysis: King's Circle

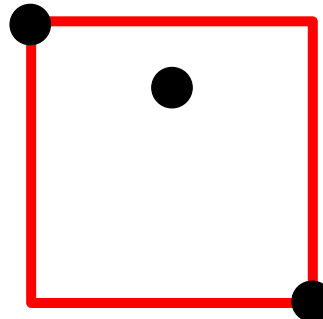
Let's set the story aside and consider the underlying geometry problem: given N points in the 2D plane, how many triplets of points have an axis-aligned square that goes through them?

It is natural to first tackle this subproblem: given three points in the 2D plane, is there an axis-aligned square that goes through those three points?

After trying different sets of points, it is not too difficult to come up with the key restriction. Consider the axis-aligned bounding box (that is, the rectangle which contains all the points which is as small as possible) for the three points. An axis-aligned square that goes through the three points exists if and only if all three points lie somewhere on the edges of the bounding box (see below).



Square exists



Square does not exist

This idea is simple enough to start working with even without a thorough proof (which we will get back to later).

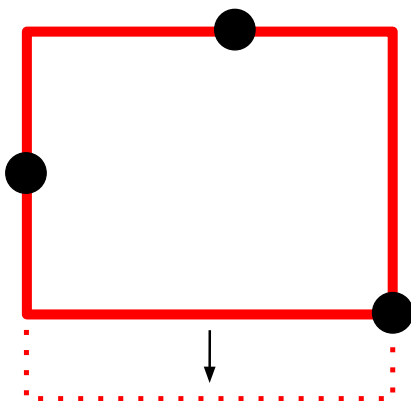
A bounding box around two points will always have those points at opposite corners. The bounding box might have zero area (if both points are horizontally or vertically aligned), but that's fine. This gives us enough information to solve the Small dataset: If two points are chosen

as the corners of a bounding box, then these two points will form a valid triplet with any point that is **not** strictly inside the bounding box. That is, any point outside the bounding box or along its edges will form a valid triplet with these two points. This gives us an $O(N^2)$ solution: For every pair of points, count the number of points that are strictly inside their bounding box, which can be done in $O(1)$ using a 2D cumulative sum array, taking advantage of the small bounds on the coordinates in the small test set. Since every invalid triplet is counted exactly once (for any invalid triplet, exactly one of the points will be inside the bounding box defined by the other two points).

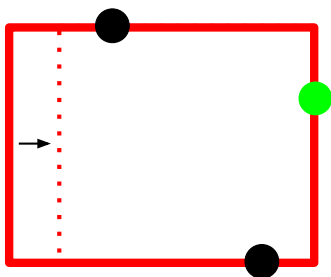
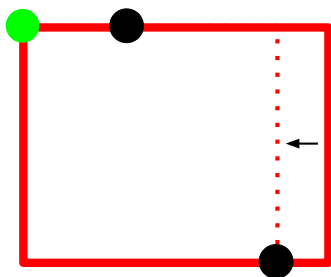
For the Large dataset, we need to turn around our thinking. Instead of fixing the bounding box and counting the number of points inside it, we will fix a point as the "inside" point, and count how many different bounding boxes contain it. Let's say we fix an inside point. Let **A** be the set of points above and to the right of our fixed point, and **B** be points below and to the left. Then there are $A \times B$ bounding boxes that contain our fixed inside point: every point in **A** forms a bounding box containing our fixed inside point with any of the points in **B**. The same can be said for the points above and to the right combined with those below and to the left.

This gives us an $O(N \log N)$ solution: for each point, count the number of bounding boxes that would contain it by finding the number of points above and to the left, above and to the right, below and to the left and below and to the right, and multiplying accordingly. Counting these points can be done in $O(\log N)$ using a linear sweep with a [range tree](#) or with a self balancing binary search tree (among other ways).

To wrap things up, we need to prove our original assertion: An axis-aligned square that goes through the three points exists if and only if all three points lie somewhere on the edges of the bounding box. The negative case is easy. If there is a point not on the edges of the bounding box, then there is no axis-aligned *rectangle* that goes through the three points, let alone a square. The affirmative case is more difficult. We will start with the bounding box and morph it into a square. If the bounding box is already a square, then we are done. Otherwise, one pair of parallel sides is longer. If either of these sides does not have a point on it *excluding the corners*, then that side can be extended until the shape is a square (see below).



So what do we do if both long sides have a point on them (not at a corner)? This is actually impossible. For a contradiction, suppose that both long sides do have a point on them, not at a corner. It doesn't matter where the third point is, there is one side with no points on it at all (not even at the corners)! This means the bounding box can be shrunk, which contradicts our definition of the bounding box being as small as possible.



Kick Start 2018 - Round B

Analysis: No Nine

No Nine: Analysis

Small dataset

To solve the Small dataset, we can check all numbers in the range $[A, B]$ and count how many of them are legal.

Large dataset

Let $f(X)$ be the number of legal numbers in the range $[0, X]$. Then the answer is $f(B) - f(A) + 1$, since A and B are both legal numbers.

To calculate $f(X)$, let $x[0], x[1], \dots, x[n-1]$ be the decimal representation of X , such that $X = \sum_{0 \leq i < n} x[i] \times 10^i$. For numbers in the range $[X - x[0], X]$, we will check each number individually to see if it is legal.

If we list numbers without the digit 9 in their decimal representations, we can find that their decimal representations are the same as listing numbers in base 9. So for numbers in the range $[0, X - x[0]]$, there are $C = \sum_{1 \leq i < n} x[i] \times 9^i$ numbers consisting only of digits in the range $[0, 8]$. According to the formula, C is divisible by 9.

For any integer Y , there is exactly one number divisible by 9 in the set $\{10Y + 0, 10Y + 1, \dots, 10Y + 8\}$. The C numbers form $C/9$ such groups, and in each group, there are exactly 8 legal numbers, so there are $8C/9$ legal numbers in the range $[0, X - x[0]]$.

Alternative solution (Dynamic Programming): To calculate $f(X)$, we can define $dp[i][j]$ to be the number of integers Y such that

- $Y < \text{floor}(X/10^i)$
- Y does not contain 9 in its decimal representation;
- $Y \equiv j \pmod{9}$.

Then $f(X) = dp[0][1] + dp[0][2] + \dots + dp[0][8] + 1$.

The [Bellman equation](#) is $dp[k-1][j] = \sum [dp[k][j']]$ for $0 \leq d \leq 8, 10j' + d \equiv j \pmod{9}$ + $\{g \mid 10\text{floor}(X/10^k) \leq g < \text{floor}(X/10^{k-1}), g \equiv j \pmod{9}\}$.

Analysis: Sherlock and the Bit Strings

Bit String: Analysis

The main challenge of finding the P -th lexicographically smallest valid string is to count the number of valid strings given a prefix requirement (i.e. the string must begin with a given prefix) and the K constraints given in the problem. The prefix requirement is necessary since we're going to fill the characters of the string one by one from the first character. Once we have solved the counting problem, the original problem becomes easier.

We can loop through each of the valid characters i in lexicographically increasing order (i.e. 0 and 1 in this problem) for the first position. Then we count the number of valid strings with prefix i . Let's say this value is X . If $X \geq P$, we know that i is the right character for the first position. Otherwise, we continue the loop and subtract X from P until we find the right character for the first position. We keep doing this for each position until the last position.

To avoid an overflow when counting the number of valid strings, we can limit our computation to maxP , where maxP is the maximum value of P (i.e. 10^{18} in this problem). In other words, whenever we find that the number of valid strings would be more than maxP , we can set it to maxP .

Small dataset

Finding the number of valid strings in the Small dataset is easier since $A_i = B_i$ in each of the constraints. It is the same as saying that the A_i -th character must be C_i . Therefore, the set of possible characters for each position is independent of what character we choose for any other position. The number of valid strings is simply 2^Y , where Y is the number of positions without a constraint. For example, the number of valid strings without a given prefix requirement is 2^{N-K} .

Large dataset

Finding the number of valid strings in the Large dataset is trickier since the set of possible characters for each position is no longer independent of what character we choose for any other position.

We can solve this dataset using a dynamic programming solution. Let us define a function $f(x, \text{last})$, where x is a $[1, N]$ integer and last is a 15-bit (15 is the maximum possible value of $B_j - A_j$) integer, is the number of ways of:

- assigning 0s and 1s to the bit string from the x -th position to the N -th position (in other words, assume that the bit string has been "filled" until the $(x - 1)$ -th position),
- the last 15 characters (i.e. from $(x - 15)$ -th position to the $(x - 1)$ -th position) are represented in last , and
- the assignment has to satisfy all the constraints i where $B_i \geq x$.

How do we calculate $f(x, \text{last})$? We loop over i in the list $[0, 1]$, the character that we consider putting in the x -th position. Together with last , we now know what the last 16 characters we have placed are. Since $B_j - A_j \leq 15$, and using the information of last 16 characters that we have placed, we can check all the constraints j where $B_j = x$.

If at least one of these constraints is violated, then i is not a valid character to be placed in on the x -th position. Otherwise, placing in i on the x -th position contributes $f(x + 1, g(last, i))$ to $f(x, last)$, where $g(last, i)$ is updating the `last` mask by ignoring the first bit of `last` (since we won't need the character from the $(x - 15)$ -th position anymore) and appending an i bit to the end.

With the prefix requirement `s`, the number of valid strings is the value of $f(\text{length}(s) + 1, \text{the last 15 characters of } s)$. Since there are N possible values for x , 2^{15} possible values for `last`, and each computation of $f(x, y)$ requires a loop for checking the constraints, this solution runs in $2^{15} \times O(N + K)$ time. Even though 2^{15} is a constant multiplicative factor that does not affect the big-O time of the solution, it is relevant to the running time, particularly because 2^{15} is much greater than the largest possible value of $N + K$, so we include it here for clarity.